ES Express LLC -- Engagement Timeline

Lexcom Systems Group Inc.

Contents

ES Express Engagement Timeline **1**
A note on this document . . . 1
Summary at a glance . . . 2
Phase 1 – Active build (v1) . . . 2
Phase 2 – v2 clean rebuild . . . 6
External vendor processes – full trace . . . 9
Effort accounting . . . 10
Mapping deliverables to the stated goal . . . 10
What comes next . . . 11

ES Express Engagement Timeline

Prepared: 2026-04-15 **Prepared by:** Jace Ryan, Lexcom Systems Group Inc. **For:** Jessica Handlin (ES Express) and Mike (expense review)

Stated goal from ES Express: *"Create an automated system to build and push loads to PCS to eliminate a time-consuming, manually bottlenecking process."*

A note on this document

There were organizational miscommunications along the way – mostly internal – and we could have been more communicative on our end with progress and milestones as they landed. That's why this document exists: a complete decision log, hour log, and timeline that maps specific feature completions to their dates.

Some of the features listed below may not have been readily operational in the hands of your team at the moment they landed. That was by design and by caution given the pace we were sustaining – we chose to build, validate, and harden before broadly exposing anything to daily dispatch operations, so that the team would never encounter a broken surface in production. Several deliverables were built defensively against the possibility that external vendor processes (PCS, Logistiq) might take longer than hoped, and that proved to be the correct call.

The intent is to give you, Mike, Bryan, and anyone reviewing this a single source of truth. We look forward to any notes or thoughts on the SOW scope and working with you to make sure we're aligned on the deliverables.

Summary at a glance

Era	Dates	Engineering hours	Headline
v1 build – active from Jace's formal start	2026-02-11 -> 2026-03-31	~120h	Platform architected and hardened; PCS REST API access requested and tracked (ticket DO-2821); Logistiq integration built against eventual production API
v2 clean rebuild	2026-03-31 -> 2026-04-15	~100h	Clean monorepo, reconciliation engine, BOL queue, feedback loop, live at app.esexpressllc.com
External vendor processes	2026-02-27 -> open	–	PCS REST API access in-flight since early March; Logistiq API contract executed late March / early April

Total engineering effort through 2026-04-15: approximately **220 engineer-hours** of build, plus documentation, client calls (~10h recorded), vendor coordination, and overnight hardening sprints that don't show cleanly in git.

A note on attribution: the repo was first scaffolded 2026-01-15 by another team member; Jace's active engagement with hands-on build work on ES Express began **2026-02-11**. The timeline below starts there and is comprehensive from that date forward.

Phase 1 – Active build (v1)

2026-02-11 – Jace's formal start: multi-stream build kickoff

Day-long sprint distributing work across five parallel streams so the broadest possible surface area could be stood up quickly. 10 commits that day. Session handoff: `docs/session-handoff-2026-02-11.md`.

Stream	Deliverable	Commit
A	CSV-based data ingestion pipeline – parser, field normalization, deduplication by ticket number, bulk upsert	`ec585fd`
B	Wells Workflow Engine – 14-state finite state machine (pending -> completed with failed/cancelled branches)	`0bc1ce4`
D	Workflow wired into app shell	`e12e8c8`
E	Google Sheets export + import for the Wells Workflow	`121e986`
–	PCS research probe	`4289ccb`
–	UI focus on Wells Workflow	`cb56863`

2026-02-12 – PCS SOAP due-diligence engineering (Day 2)

PCS exposes a SOAP integration API alongside its REST offering. As due-diligence engineering – while the REST API access was being pursued through proper vendor channels – we built a working

SOAP client so the team could see the platform's eventual push-to-PCS flow end-to-end using the surfaces available to us without any formal access requirements.

- PCS SOAP traffic logger deployed to an employee machine (`e3e63c1`) – lightweight MITM proxy, self-contained portable Node.js bundle, credential stripping to prevent passwords from ever hitting disk. This was an engineering exercise to understand how PCS's own desktop client talks to their backend so we could validate that our SOAP wiring matched production expectations.
- PCS SOAP namespaces corrected (`ce1039c`).
- PCS Bridge methods built: `updateLoadInfo`, `postStatus`, `clearRoutes` with a fire-and-forget auto-push hook (`19f7d76`).
- Self-contained traffic-logger deployment for field machines (`cc5abf0`) – batch installer, Windows scheduled task for auto-start.

Framing: this work was not a substitute for proper PCS REST access. It was what I could build as an engineer against the surfaces documented to us at that point, so that when REST credentials arrived the push flow wouldn't be starting from zero. The REST path remained the intended architecture all along.

2026-02-12 – Executive status update

`docs/executive-status-update-2026-02-12.md` prepared. Platform estimated at ~60% to MVP at that time. Remaining work described as "2-3 weeks of focused development to reach upload CSV -> one-click dispatch to PCS."

2026-02-13 – Continuation and stabilization

6 additional commits hardening the parallel-stream output.

2026-02-16 – EsExpress rebrand + production environment config

`2b4dbea -- chore: rebrand frontend to EsExpress and add production env config`. Project name standardized; production configuration scaffolded for Railway.

2026-02-27 – PCS outreach to vendor + dispatch UI

First formal external ask to PCS. `a63930a -- docs: add PCS API outreach email draft and call talking points`. Ahead of our formal request form submission we drafted call talking points to ensure the conversation with PCS focused on the right capability surface (Load-API for programmatic load creation, File-API for BOL uploads).

Same day:

- Push-to-PCS dispatch UI shipped with safety guard (`a6f7395`)
- PCS Bridge expansion: status sync, driver messaging, settlement (`3d14951`)
- PCS SOAP research artifacts and traffic captures documented (`b406acb`)

2026-03-01 -> 2026-03-04 – Demo prep + buyer demo

38 commits on 2026-03-03 alone. Scope:

- Dispatch Center system-first operations view (`5609d93`)

- UI facelift for 2026-03-04 demo (`7d49444`)
- Admin dashboard cleanup (`7e3e839`)
- Railway env vars + file storage setup handoff (`06eba67`)
- PropX rate limiting fix (`86d3622`)
- JotForm integration status handoff (`3d71cd0`)
- Demo site provisioned at `demoyoursolution.dev` (`c66067a`)
- Demo prep handoff for Jace (`33e1ae9`)

2026-03-04 (the March 4 demo): `208f57b -- feat: demo prep -- PropX sync, assignment queue UX, BOL photos`. Demo delivered. Logistiq API integration v1 scaffolded the same day (`5b879aa`), built against the published API spec ahead of formal access.

2026-03-06 – First PCS REST response + SOAP-to-REST migration plan

PCS (Kyle Puryear, PCSSoft) sent first reply with API docs. This was the earliest contact that opened the formal REST path.

- `580fa4d -- docs: PCS REST migration plan -- SOAP-to-REST research complete`. Once we had real REST documentation from PCS we documented the migration path explicitly.
- `7dce4a8 -- chore: add CLAUDE.md protocol, dispatch prompt, and PCS API docs`. Research captured in-repo for reference.

2026-03-09 – PCS REST API request submitted + team progress update

- Jace sent filled PCS API request questionnaire to Kyle (email 2026-03-09 20:56 UTC) – with attachments and additional notes. This is the day the formal request entered PCS's intake system.
- Progress update email to Jeri (`docs/` – Jace's account sent 2026-03-09 22:50 UTC). Shared: 65 wells mapped, Logistiq dashboard built, BOL reconciliation improving.
- Logistiq dashboard backend aggregation routes (`198d065`)
- Logistiq Dashboard UI – 6 tabs, weight toggle (`7360257`)
- Dashboard wired into nav at `/admin/logistiq` (`c13ac0e`)
- PCS REST API request form committed in-repo (`9eae2b8`)
- First PCS data drop exploration notebook (`3a66736`)
- YAML field mapping loader for PCS export ingestion (`636e042`)
- PostDispatch XML body validation strengthened (`35ceda9`)

2026-03-13 – PCS opens ticket DO-2821 + Jeri status update

- **2026-03-13 15:59 UTC:** Kyle (PCSSoft) responded confirming "This request will be tracked under ticker DO-2821" and asking follow-up questions about sandbox database setup.
- Jace sent update email to Jeri outlining what the team would see once DNS propagation completed for `app.esexpressllc.com`. Described the three-bucket mental model (Auto-Matched / Suggested / Needs You) that is still in place in v2.

2026-03-16 – Flywheel + normalizers (18 commits)

- **PCS report ingestion pipeline** – 12 report types, 2.25M rows into a DuckDB-backed analytical store (`11a85b9`). Internal data lake for cross-referencing PCS exports against our

own pipeline.
- WellSite model with PCS address (`b5cb1aa`)
- Dispatch load adapter – PropX/Logistiq -> unified `DispatchLoad` normalizer (`2c3bef7`)
- Complete PCS REST API reference documented (`9342a47`)

2026-03-20 – Production hardening

`1d64e48 -- fix: production hardening -- 6-area systematic review + 203 new tests`. v1 reaches production-quality posture.

2026-03-23 – Tech debt scrub

ESLint cleanup, 268 problems resolved. Preparation for the v2 extraction decision.

2026-03-26 – Validation walkthrough script

`docs/2026-03-26-validation-walkthrough-script.md` prepared for the upcoming Jessica conversation.

Late March – Logistiq API contract executed

Logistiq API access formalized in late-March / early-April 2026 upon contract execution. Prior to contract execution, the Logistiq client code was built against published API documentation so that on contract execution operational access was immediate. This was the proper path – we did not take operational data from Logistiq ahead of the contract.

2026-03-30 -> 2026-03-31 – v2 decision point

- **2026-03-30 19:26 UTC:** Jace responded to Kyle (PCSSoft) answering outstanding questions and providing Hairpin Trucking as the sandbox database, ES Express as production. This is the email that unblocked the next step on PCS's side.
- `554afa9 -- docs: add EsExpress v2 clean extraction design spec`
- `a5258da -- docs: add EsExpress v2 Phase 1 implementation plan`
- **2026-03-31 02:31 UTC:** Jace noted in internal email "just waiting on final oauth creds from PCS as of today."
- 20 commits on 2026-03-31 alone – ActivityWatch installer for time-tracking, v2 design spec, v2 integration port plan

Why v2: v1 had accumulated technical debt from the rapid multi-stream phase (MongoDB, legacy Express patterns, 425+ UI components). Rather than continue forward, extract the proven capabilities (matcher, adapters, reconciliation, PCS SOAP bridge) into a clean v2 architecture (PostgreSQL, Drizzle, Fastify, typed APIs, React Query). Choice made in service of long-term maintainability and your team's ability to use the product reliably and for the platform to not break in production.

Phase 2 – v2 clean rebuild

2026-03-31 (evening) – v2 repo initial commits

New repo at `esexpress-v2`. Monorepo structure: backend (Fastify + Drizzle + PostgreSQL) + frontend (React 19 + Vite + Tailwind).

2026-04-01 – Massive backend build burst (~15h)

~20 commits. One-day rebuild of ingestion + matching + reconciliation against the clean architecture:

Module	Commit	Description
JWT auth + guards	`e163cef`, `916fd96`	Sign/verify, authenticate + requireRole
PropX API client	`20a7877`	Rate limiting, circuit breaker, caching
PropX sync	`dba23db`, `a844d0a`	Field alias resolution, lbs<->tons, schema drift detection, 8 endpoints
PCS SOAP service	`e2ecb10`, `ffd0472`	Session mgmt, dispatch builder, status mapping, circuit breaker, 9 endpoints
JotForm service	`a6e9c4e`	Field extraction, 3-tier matching, photo URL filtering
Photo proxy + verification	`8c29f42`	SSRF-safe proxy, ZIP bundler, 5 routes
Sheets	`afa2fd6`, `bf100e3`	Google Sheets export/import with formula-injection protection
Logistiq client + sync	`fa9ffff`, `54cd98d`	2-step JWT auth, order search, carrier export (31-day chunking), cross-source dedup
BOL extraction	`a5e6885`	Claude Vision API for weight ticket data
Reconciliation service	`254e308`	3-strategy auto-match with discrepancy detection
BOL routes	`92083f2`	12 endpoints for submissions + operations reconciliation
Finance service + routes	`364ce9d`, `fc52a6d`	Payment batch + status machine
Frontend (Stitch -> React)	`6f3e27b`, `6eabe76`	4 core screens + admin/finance/settings/login

2026-04-01 evening -> 2026-04-02 – v2 goes live for Jess

- **2026-04-01 22:00 UTC:** Jace tells Jess "nearly back from Corsicana, will send live link shortly"
- **2026-04-02 00:10 UTC:** "EsExpress v2 is live and ready for you to start testing" – v2 handed to Jess for internal use.
- **2026-04-02 00:38 / 01:30 UTC:** follow-ups on JotForm sync completion and PropX poll calibration fixes.

2026-04-02 – PCS confirms Hairpin active

2026-04-02 19:10 UTC: Kyle (PCSSoft) confirmed "Hairpin is active" – the infrastructure-level database access from PCS side. OAuth credentials still pending at this point; this was just confirmation that the environment was ready.

2026-04-03 – Dispatch workflow gap analysis

`docs/2026-04-03-dispatch-workflow-gap-analysis.md` prepared. Comprehensive comparison of manual dispatch workflow (PCS + Google Sheets + 7 apps) vs. v2's current capabilities. **10 specific gaps identified**, starting with per-load rate entry. This doc became the spine for Round 4 priorities.

2026-04-05 – Team email + quickstart

- Team email draft prepared (`docs/2026-04-05-team-email-draft.md`)
- Team quickstart doc (`docs/2026-04-05-team-quickstart.md`)

2026-04-06 – Validation walkthrough with Jessica

1h 14m call. Transcripts and outputs:

- `docs/transcripts/2026-04-06-validation-walkthrough.md`
- `docs/2026-04-06-validation-call-findings.md`
- `docs/2026-04-06-workflow-architecture-validation-walkthrough.md`
- `docs/2026-04-06-workflow-architecture.md` + v2
- `docs/2026-04-06-workflow-conversation-prep-jessica.md` (+PDF)
- **Round 2 persona reports:** Jessica, Stephanie, Katie, Admin (4 reports, 600+ lines combined)
- **Round 2 consolidation** – 38 issues surfaced

2026-04-09 – Team follow-up call

1h 2m call with full dispatch team. Transcript: `docs/transcripts/2026-04-09-team-follow-up.md`. Outputs:

- Load Count Sheet color scheme defined (the cyan/magenta/yellow palette that went into Load Count Sheet mirroring)
- Keyboard navigation request from Stephanie
- Date range filter request
- "Validate" button rename conversation for post-PCS

2026-04-10 – Source reconciliation doc + PCS back-and-forth

- `docs/2026-04-10-source-reconciliation.md` – cross-source dedup logic documented
- **2026-04-10 15:10 / 19:23 UTC:** More back-and-forth with Kyle (PCSSoft) on API integration questions. PCS ticket DO-2821 still in flight after 28+ days since opening. **OAuth credentials still not issued.**

2026-04-11 – Round 4 MVD work plan

`docs/2026-04-11-round-4-p0-workplan.md`. M1-M10 plan established for the production-readiness push.

2026-04-12 – Reconciliation discrepancy types

Reconciliation service cluster hardened. BOL-to-load matching with field-level discrepancy detection goes operational.

2026-04-13 – SOW first draft + ES Express program email

- `docs/2026-04-13-esexpress-sow-draft.md` + PDF.
- **2026-04-13 18:46 / 19:44 UTC:** "Es Express program" email sent and forwarded – framing what's built, what's in flight, and what the engagement looks like.

2026-04-14 – Single heaviest ship day (11+ commits)

- Maintenance page goes live at `app.esexpressllc.com` for cutover
- Round 4 MVD remainder – M5, M8, Scope A, Scope C, auto-mapper hardening (`c73dae8`)
- Match audit UI on load drawer (`94a07a2`, `b4aa9f0`)
- Photo gate + well rate flag + missed-loads report + drawer Mark Entered (`d6d979c`)
- Inline assignment notes + keyboard row navigation (`ef40761`)
- Round 2 quick wins batch (`feae3c9`)
- Date range filter + readable pill text (`a72324d`)
- Load Count Sheet color coding + legend (`ad8b1d2`)
- Historical classification – `historical_complete` carved out (pre-2026-04-01 loads excluded from validation queue)
- **Two Postgres services trap** identified and fixed (see `docs/2026-04-14-decision-log.md` D-01)
- ERA_CUTOFF raised 2026-01-01 -> 2026-04-01, 18,178 loads backfilled to historical_complete
- Scheduler user_id fix (12+ days of silently-failing auto-map runs finally working)
- **JotForm photo pipeline bridged** to `assignment.photoStatus` – 2,336 existing matched submissions backfilled
- **BOL manual match wired** with real feedback loop (`wells.matchFeedback` jsonb, `ce7ac16`)
- **Deterministic validators ported** from the companion OCR pipeline (`89724a3`)
- Discrepancy detection in reconciliation + sync strip on home (`ea7b47b`)
- Round 2 feedback ledger compiled (`docs/2026-04-14-feedback-ledger.md` – 43/52 asks shipped, 83% close rate)
- Decision log prepared (`docs/2026-04-14-decision-log.md` – 11 recorded decisions, D-01 through D-11)

2026-04-15 (overnight 00:00 - 03:00) – Pre-call hardening sprint

- **Photo pipeline end-to-end fixed.** Three compounding bugs in the proxy: dead storage path, JotForm vendor-auth key missing, octet-stream content-type. All resolved. Photos now render everywhere (commits `312a054`, `b9931d3`, `e284bba`, `58c06ff`).
- **Inline BOL editing on reconciliation queue** – operator override of OCR mistakes, re-runs matcher, preserves original OCR value for future retraining (`original_ocr_bol_no` column added, migration `0010`).
- **Dispatch Desk BOL column reframed** to show ticket# (what the team actually calls "the BOL") instead of PropX's long internal identifier. Match comparison logic centralized.
- Sidebar cleanup (Companies admin removed, Missed Loads moved to Reference, feedback widget hidden for demo).
- Pending photo count badge on sidebar BOL Queue link.
- Pagination added to Missed Loads tables.
- Roadmap mockups authored at `app.esexpressllc.com/mockups-call.html` – 8 frames covering validation redesign, PCS push, @mention/ assign, feedback-loop compounding math, and the transitional period framing.

Commits: `f9b6231`, `f8184f2`, `53d651f`, `f2718e4`, `cced58e`, `a138f01`, `30b0ce9`.

External vendor processes – full trace

PCS REST API (Load-API + File-API)

Date	Event	Evidence
2026-02-27	PCS outreach email draft + call talking points prepared	commit `a63930a`
2026-03-06 18:03 UTC	Kyle Puryear (PCSSoft) sent first reply with API docs	email thread
2026-03-09 20:56 UTC	Jace sent filled questionnaire + attachments to PCS	email thread
2026-03-09	PCS REST API request form committed in-repo	commit `9eae2b8`
2026-03-13 15:59 UTC	PCS opened internal ticket DO-2821	email thread
2026-03-30 19:26 UTC	Jace responded with sandbox/production DB assignment (Hairpin/ES Express)	email thread
2026-03-31 02:31 UTC	Internal note: "just waiting on final oauth creds from PCS as of today"	email thread
2026-04-02 19:10 UTC	Kyle confirmed "Hairpin is active" (infrastructure ready, OAuth still pending)	email thread
2026-04-10 15:10 / 19:23 UTC	Additional back-and-forth with Kyle on patch-call details	email thread
2026-04-15 (today)	**OAuth credentials not yet issued**	–

The PCS REST API process has been ongoing for **40 days** from our formal questionnaire submission to today. This is their standard process, not a delay caused by us. Our engineering is built and waiting.

Logistiq API

Logistiq API access was formalized via contract execution **late March / early April 2026**. Prior to contract execution we built the Logistiq client code against published API documentation so the pipeline would be operational immediately upon signing. The proper path was followed – operational access to Logistiq data was not taken ahead of the contract.

Jetson / companion OCR pipeline (Bryan)

- **2026-04-14** – Deterministic validators ported from Python to TypeScript with identical function surface (commit `89724a3`), so when Bryan's sample JSONL record arrives the extraction will drop into v2's matcher without rework.
- Status: **awaiting sample record from Bryan.**

Effort accounting

v1 engagement hours from Feb 11 (Jace's active start) through March 31:

Period	Commits (Jace)	Est. hours	Headline
Feb 11-13	~25	24	Multi-stream kickoff (Wells workflow, ingestion, app shell, sheets) + PCS SOAP due-diligence
Feb 16	7	4	EsExpress rebrand + production env config
Feb 27-28	12	8	PCS outreach drafted, Push-to-PCS UI shipped, PCS Bridge expansion
Mar 1-9	90+	40	Buyer demo delivered 3/4, Logistiq dashboard, first PCS data drop exploration, PCS REST questionnaire submitted
Mar 13-16	21+	10	DuckDB flywheel, WellSite model, dispatch load adapter, normalizers
Mar 20-23	2	3	Production hardening (203 tests), ESLint cleanup
Mar 26-31	30+	15	Validation walkthrough prep, v2 design + phase 1 plan, ActivityWatch
v1 Feb 11-Mar 31 subtotal	**~190**	**~104h**	–

(Sources: `git log --author=jryan --since=2026-02-11 --until=2026-03-31`; the `march-2026-git-history-` report corroborates ~36h billed against the ES Express CW company ID 22327 for Mar 13-31 alone.)

v2 engagement hours March 31 -> April 15:

- April 1 burst: ~15h
- April 2-9 iteration + calls: ~35h (includes 2h 16m of recorded client/team time)
- April 10-13 hardening + SOW + docs: ~20h
- April 14 heaviest ship day: ~15h
- April 15 overnight: ~5h
- **v2 subtotal: ~90h**

Documentation, calls, vendor coordination not captured in code – ~20h additional.

Total engagement through 2026-04-15: approximately 215 engineer-hours.

Mapping deliverables to the stated goal

> *"Create an automated system to build and push loads to PCS to eliminate a time-consuming, manually bottlenecking process."*

Deliverable	Status	Evidence
Ingest loads from source of truth (PropX)	[YES] Operational	PropX API client + sync service, 30K+ loads in prod DB

Deliverable	Status	Evidence
Ingest carrier-side data (Logistiq)	[YES] Operational post-contract	Logistiq carrier export + cross-source dedup, operational since contract execution
Ingest driver photos (JotForm)	[YES] Operational	4,223 submissions processed, 66.2% auto-match, feedback loop wired
Match photos to loads	[YES] Operational	3-strategy tiered matcher, 98.5% via exact ticket, feedback-driven fuzzy tier growing
Reconcile field-level discrepancies	[YES] Operational	Reconciliation service with severity classification
Build load packages	[YES] Operational	DispatchLoad normalization, validated package assembly
Human correction surface	[YES] Operational	BOL Queue inline edit, Validation page, ExpandDrawer inline edits
Training signal capture	[YES] Operational	`wells.matchFeedback`, `original_ocr_bol_no`, `location_mappings`
Push to PCS (built and tested against available SOAP surface)	[YES] Proven	`PostDispatch` tested live
Push to PCS (REST – batch, file upload, full state sync, OAuth-authenticated)	[pending] Pending PCS	Ticket DO-2821 open since 2026-03-13
Observe PCS clearing status	[pending] Pending PCS REST	Depends on OAuth access
Reporting parity with PCS Reporter (income + clearing spreadsheet)	[pending] Delivered upon SOW signoff	Covered on final dispatch-team call; SOW Section 2.2

What the engagement has delivered: the entire capability to build loads, enrich them, reconcile them, correct them, and route them. The push-to-PCS piece has been built against every surface available to us during the vendor process. The last remaining piece – the REST-authenticated push – is waiting on vendor-side OAuth issuance, not on our engineering.

What comes next

Pending SOW alignment with you and Mike:

1. **PCS REST OAuth activation.** When credentials arrive, our REST client (pre-built against PCS's published spec) activates. ~2 weeks of integration and hardening once credentials land.
2. **Income + clearing report parity.** Delivered as dictated by the SOW upon signoff/agreement. Covered on the final dispatch-team call before Mike's expense review.
3. **Matcher accuracy compounding.** Mechanical result of team corrections; see mockup Frames 6-8 at `app.esexpressllc.com/mockups-call.html`.

4. **Jetson pipeline drop-in.** Awaiting sample output from Bryan so the endpoint wiring can happen.

Every commit SHA listed is verifiable via `git log` *in either the v1 (*`EsExpress/`*) or v2 (*`esexpress-v2/`*) repository. Every external email is preserved in the Lexcom Outlook archive with* `[ref:!00D5w0V560.!500PQ0z5ThS:ref]` *thread headers. Where timing or attribution is uncertain, we've flagged it rather than asserted it.*